



POLITÉCNICA

EXAMEN FINAL FICHEROS/SINCRO. CURSO 2025/2026 (JULIO)

Fernando Pérez Costoya
Curso 2026/27

- **Partición 4GiB**
- **Agrupación 4KiB**
- **Direcciones de agrupación de 32 bits**
- **Inodos de 128B**
- **12 punteros directos, un indirecto simple, uno doble y uno triple**
- **Dimensionado suponiendo que tamaño medio de fichero es 1 agrupación**

Cuestiones

- a) **¿% ocupado por metainformación obviando sector de arranque, superbloque y directorios?**
- b) **¿Tamaño de un fichero que usa?**
 - **punteros directos**
 - **agrupación de indirección simple**
 - **2 primeras entradas completas de la agrupación de indirección doble**
- c) **Complete programa que da vuelta a fichero (intercambia byte 0 y último...)**

¿% ocupado por metainformación obviando sector de arranque, superbloque y directorios?

mapa de bits de agrupaciones + **mapa de bits de inodos** + **zona de inodos**

1. Cálculo de cuántas agrupaciones ocupa el mapa de bits de agrupaciones

- n.º de agrupaciones en la partición = tamaño disco/tamaño agrupación
 - $4\text{GiB}/4\text{KiB} = 2^{32}/2^{12} = 2^{20}$ agrupaciones
- n.º bits en mapa de bits de agrupaciones = 1 bit por cada agrupación = 2^{20} bits
- n.º bits caben en una agrupación = $4\text{KiB} * 8 = 2^{15}$ bits
- n.º agrupaciones mapa bits agrupaciones = bits del mapa/bits por agrupación
 - $2^{20} / 2^{15} = 2^5$ agrupaciones

¿% ocupado por metainformación obviando sector de arranque, superbloque y directorios?

mapa de bits de agrupaciones + mapa de bits de inodos + zona de inodos

2. Cálculo de cuántas agrupaciones ocupa el mapa de bits de inodos

- tamaño medio de un fichero es igual al de una agrupación
 - va a haber tantos inodos como agrupaciones
 - $n.º$ de inodos en partición = $n.º$ de agrupaciones en partición = 2^{20} inodos
- tamaño de este mapa será igual que el del anterior:
 - $n.º$ agrupaciones mapa bits inodos = $n.º$ agrupaciones mapa bits agrupaciones
 - 2^5 agrupaciones

¿% ocupado por metainformación obviando sector de arranque, superbloque y directorios?

mapa de bits de agrupaciones + mapa de bits de inodos + zona de inodos

3. Cálculo de cuántas agrupaciones ocupa la zona de inodos

- n.º de bytes en la zona de inodos = n.º inodos * tamaño del inodo
 - $2^{20} * 128 = 2^{27}$ bytes
- n.º agrupaciones zona de inodos = bytes zona inodos/bytes por agrupación
 - $2^{27} / 2^{12} = 2^{15}$ agrupaciones

Por tanto, % de ocupación de la metainformación:

- $(2^5 + 2^5 + 2^{15}) / 2^{20} = 3,13\%$
- NOTA: 128,25 MB de 4GiB

¿Tamaño de fichero que usa punteros directos, agrupación de indirección simple, 2 primeras entradas completas de la agrupación de indirección doble?

- Tamaño de fichero → Recuento de punteros usados por el fichero
- Antes, n.º de punteros en 1 agrupación = tamaño agrupación/tamaño puntero
 - $4\text{KiB}/4\text{B} = 2^{12}/2^2 = 1024$ punteros
- Recuento de punteros:
 - Todos los punteros directos: 12
 - Todos los punteros de la agrupación de indirección simple: 1024
 - Todos los punteros de la agrupación apuntada por la 1ª entrada de la agrupación de indirección doble: 1024.
 - Todos los punteros de la agrupación apuntada por la 2ª entrada de la agrupación de indirección doble: 1024.
- Por tanto, el fichero ocupa 3084 ($12 + 3 * 1024$) agrupaciones.
- Última agrupación puede contener un solo byte o estar completamente usada
- Fichero puede tener un tamaño entre:
 - $3083 * 4096 + 1 = 12\,627\,969$ bytes
 - $3084 * 4096 = 12\,632\,064$ bytes.

```
int main(int argc, char *argv[]) {
    char c1, c2; struct stat st;
    int fd = open(argv[1], O_RDWR);
    fstat(fd, &st);
    for (size_t i=0; i < st.st_size/2; i++) {
        read(fd, &c1, 1); // PUNTO1: lee el siguiente carácter de la primera mitad del fichero
        read(fd, &c2, 1); // PUNTO2: lee el carácter correspondiente de la segunda mitad del fichero
        write(fd, &c1, 1); // PUNTO3: escribe el carácter de la primera mitad en el lugar correspondiente de la segunda
        write(fd, &c2, 1); // PUNTO4: escribe el carácter de la segunda mitad en el lugar correspondiente de la primera
    }
    close(fd);
    return 0;
}
```

```
int main(int argc, char *argv[]) {
    char c1, c2; struct stat st;
    int fd = open(argv[1], O_RDWR);
    fstat(fd, &st);
    for (size_t i=0; i < st.st_size/2; i++) {
        read(fd, &c1, 1); // PUNTO1: lee el siguiente carácter de la primera mitad del fichero
        lseek(fd, -(i+1), SEEK_END);
        read(fd, &c2, 1); // PUNTO2: lee el carácter correspondiente de la segunda mitad del fichero
        lseek(fd, -1, SEEK_CUR);
        write(fd, &c1, 1); // PUNTO3: escribe el carácter de la primera mitad en el lugar correspondiente de la segunda
        lseek(fd, i, SEEK_SET);
        write(fd, &c2, 1); // PUNTO4: escribe el carácter de la segunda mitad en el lugar correspondiente de la primera
    }
    close(fd);
    return 0;
}
```